

Thiết kế phân tầng Exception chuẩn Production trong Spring Boot

1. Mục tiêu

Trong hệ thống production, exception không nên được throw/catch tùy tiện ở mọi nơi. Cần phân tầng rõ ràng để:

- Controller không biết lỗi kỹ thuật bên dưới như `SQLException`, `IOException`, `JsonProcessingException`.
- Service chỉ xử lý lỗi nghiệp vụ và orchestration.
- Infrastructure/Adapter chịu trách nhiệm wrap lỗi kỹ thuật.
- `GlobalExceptionHandler` là nơi duy nhất convert exception thành HTTP response.
- Transaction rollback đúng.
- Log đủ thông tin để debug production.
- Không expose stack trace, SQL, internal URL, SDK exception ra client.

2. Các tầng chính trong ứng dụng

Một ứng dụng Spring Boot production thường có các tầng:

```
Client
|
v
Controller Layer
|
v
Application Service Layer
|
v
Domain Layer
|
v
Infrastructure Layer
|
v
Database / External Service / File System / Message Broker
```

Mỗi tầng nên có trách nhiệm exception khác nhau.

3. Nguyên tắc tổng quát

3.1. Exception nên được xử lý ở đúng tầng

Không phải exception nào cũng catch ngay tại nơi phát sinh.

Nguyên tắc:

```
Layer phát sinh lỗi kỹ thuật
↓
Wrap thành application exception
↓
Service quyết định flow nghiệp vụ
↓
GlobalExceptionHandler map sang HTTP response
```

Ví dụ:

```
SQLException
→ DatabaseOperationException

IOException
→ FileProcessingException

JsonProcessingException
→ JsonParsingException

Payment SDK Exception
→ ExternalServiceException

Order already paid
→ BusinessException / OrderAlreadyPaidException
```

3.2. Không để exception kỹ thuật leak lên controller

Không nên:

```
@PostMapping("/orders/import")
public ApiResponse<Void> importOrders(MultipartFile file)
    throws IOException, SQLException {
    orderService.importOrders(file);
}
```

```
        return ApiResponse.success(null);
    }
```

Nên:

```
@PostMapping("/orders/import")
public ResponseEntity<ApiResponse<Void>> importOrders(
    @RequestParam MultipartFile file
) {
    orderService.importOrders(file);
    return ResponseEntity.ok(ApiResponse.success("Import completed", null));
}
```

Controller không cần biết file đọc lỗi, database lỗi hay parser lỗi.

3.3. GlobalExceptionHandler là điểm cuối để convert exception thành response

Không nên trả response lỗi thủ công ở nhiều nơi.

Không nên:

```
try {
    orderService.pay(orderId);
} catch (OrderAlreadyPaidException e) {
    return ResponseEntity.badRequest().body(...);
}
```

Nên:

```
@PostMapping("/{id}/pay")
public ResponseEntity<ApiResponse<OrderResponse>> payOrder(
    @PathVariable Long id
) {
    OrderResponse response = orderService.pay(id);
    return ResponseEntity.ok(ApiResponse.success(response));
}
```

Exception sẽ được xử lý bởi:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

4. Phân loại exception theo tầng

4.1. Controller Layer Exception

Controller layer thường xử lý:

- Request body sai JSON.
- Request param thiếu.
- Path variable sai type.
- Validation fail.
- Multipart file invalid.
- Authentication/authorization exception.

Ví dụ exception thường gặp:

```
MethodArgumentNotValidException
ConstraintViolationException
HttpMessageNotReadableException
MissingServletRequestParameterException
MethodArgumentTypeMismatchException
MaxUploadSizeExceededException
AccessDeniedException
AuthenticationException
```

Controller không nên throw exception kỹ thuật từ tầng dưới.

4.2. Application Service Layer Exception

Service layer xử lý:

- Business rule.
- Use case flow.
- Transaction.
- Validate trạng thái nghiệp vụ.
- Orchestrate repository, external client, event publisher.

Exception ở tầng này thường là:

```
BusinessException
ResourceNotFoundException
ConflictException
InvalidStateException
PermissionDeniedException
```

Ví dụ:

```
if (order.isPaid()) {
    throw new OrderAlreadyPaidException(order.getId());
}
```

Service layer không nên throw:

```
IOException
SQLException
JsonProcessingException
PaymentGatewayRawException
KafkaException trực tiếp
```

Nếu gặp lỗi từ tầng dưới, lỗi đó nên đã được wrap thành application exception.

4.3. Domain Layer Exception

Domain layer chứa business invariant cốt lõi.

Ví dụ:

```
Order cannot be paid twice
Order cannot be cancelled after shipped
Balance cannot be negative
Invalid order status transition
```

Exception domain nên độc lập với Spring, HTTP, database.

Ví dụ:

```
public class InvalidOrderStateException extends DomainException {
    public InvalidOrderStateException(String message) {
        super(message);
    }
}
```

```
}  
}
```

Domain layer không nên biết:

```
HttpStatus  
ResponseEntity  
JpaRepository  
WebClient  
KafkaTemplate  
ObjectMapper
```

4.4. Infrastructure Layer Exception

Infrastructure layer là nơi làm việc với kỹ thuật bên ngoài:

- Database.
- File system.
- Redis.
- Kafka.
- RabbitMQ.
- Payment gateway.
- Email provider.
- Object storage.
- REST client.
- Elasticsearch.

Exception ở tầng này thường là checked/technical exception:

```
IOException  
SQLException  
DataAccessException  
TimeoutException  
JsonProcessingException  
WebClientException  
FeignException  
KafkaException  
ElasticsearchException
```

Infrastructure layer nên catch lỗi kỹ thuật và wrap thành application exception.

Ví dụ:

```

try {
    return paymentGateway.charge(request);
} catch (SocketTimeoutException e) {
    throw new ExternalServiceTimeoutException("PAYMENT_GATEWAY", e);
} catch (IOException e) {
    throw new ExternalServiceException("PAYMENT_GATEWAY", e);
}

```

5. Sơ đồ phân tầng exception

Controller Layer

- Validate request
- Không catch business exception
- Không catch technical exception
- Để GlobalExceptionHandler xử lý

↓

Application Service Layer

- Throw BusinessException
- Throw NotFoundException
- Throw ConflictException
- Quản lý transaction
- Không expose IOException/SQLException

↓

Domain Layer

- Throw DomainException
- Bảo vệ business invariant
- Không phụ thuộc Spring/HTTP

↓

Infrastructure Layer

- Catch IOException, SQLException, SDK Exception
- Wrap thành ExternalServiceException, FileProcessingException, DatabaseOperationException

↓

GlobalExceptionHandler

- Convert exception thành ErrorResponse

- Map HTTP status
- Add traceId
- Log đúng level

6. Cấu trúc package đề xuất

```
src/main/java/com/example/project
├── common
│   ├── exception
│   │   ├── ErrorCode.java
│   │   ├── BaseException.java
│   │   ├── BusinessException.java
│   │   ├── DomainException.java
│   │   ├── ResourceNotFoundException.java
│   │   ├── ConflictException.java
│   │   ├── ExternalServiceException.java
│   │   ├── ExternalServiceTimeoutException.java
│   │   ├── FileProcessingException.java
│   │   ├── DatabaseOperationException.java
│   │   └── GlobalExceptionHandler.java
│   └── response
│       ├── ApiResponse.java
│       ├── ErrorResponse.java
│       └── FieldErrorResponse.java
├── order
│   ├── controller
│   ├── application
│   │   └── OrderService.java
│   ├── domain
│   │   ├── Order.java
│   │   └── exception
│   │       ├── OrderAlreadyPaidException.java
│   │       └── InvalidOrderStateException.java
│   ├── infrastructure
│   │   ├── OrderRepository.java
│   │   └── PaymentClient.java
│   └── dto
└── payment
    ├── application
```


7. BaseException

```
package com.example.common.exception;

public abstract class BaseException extends RuntimeException {

    private final ErrorCode errorCode;

    protected BaseException(ErrorCode errorCode) {
        super(errorCode.getMessage());
        this.errorCode = errorCode;
    }

    protected BaseException(ErrorCode errorCode, String message) {
        super(message);
        this.errorCode = errorCode;
    }

    protected BaseException(ErrorCode errorCode, String message, Throwable
cause) {
        super(message, cause);
        this.errorCode = errorCode;
    }

    protected BaseException(ErrorCode errorCode, Throwable cause) {
        super(errorCode.getMessage(), cause);
        this.errorCode = errorCode;
    }

    public ErrorCode getErrorCode() {
        return errorCode;
    }
}
```

8. ErrorCode

```
package com.example.common.exception;

import org.springframework.http.HttpStatus;

public enum ErrorCode {

    // Common
    INTERNAL_SERVER_ERROR("COMMON_500", "Internal server error",
        HttpStatus.INTERNAL_SERVER_ERROR),
    VALIDATION_ERROR("COMMON_422", "Validation failed",
        HttpStatus.UNPROCESSABLE_ENTITY),
    INVALID_REQUEST("COMMON_400", "Invalid request", HttpStatus.BAD_REQUEST),
    RESOURCE_NOT_FOUND("COMMON_404", "Resource not found",
        HttpStatus.NOT_FOUND),
    CONFLICT("COMMON_409", "Conflict", HttpStatus.CONFLICT),

    // Order
    ORDER_NOT_FOUND("ORDER_404", "Order not found", HttpStatus.NOT_FOUND),
    ORDER_ALREADY_PAID("ORDER_001", "Order has already been paid",
        HttpStatus.CONFLICT),
    ORDER_INVALID_STATE("ORDER_002", "Invalid order state",
        HttpStatus.BAD_REQUEST),

    // File
    FILE_READ_ERROR("FILE_001", "Cannot read file", HttpStatus.BAD_REQUEST),
    FILE_FORMAT_INVALID("FILE_002", "Invalid file format",
        HttpStatus.UNPROCESSABLE_ENTITY),

    // External
    EXTERNAL_SERVICE_ERROR("EXT_001", "External service error",
        HttpStatus.BAD_GATEWAY),
    EXTERNAL_SERVICE_TIMEOUT("EXT_504", "External service timeout",
        HttpStatus.GATEWAY_TIMEOUT),
    EXTERNAL_SERVICE_UNAVAILABLE("EXT_503", "External service unavailable",
        HttpStatus.SERVICE_UNAVAILABLE),

    // Database
    DATABASE_ERROR("DB_500", "Database error",
        HttpStatus.INTERNAL_SERVER_ERROR),
    DUPLICATE_RESOURCE("DB_409", "Duplicate resource", HttpStatus.CONFLICT);

    private final String code;
    private final String message;
    private final HttpStatus status;
}
```

```

    ErrorCode(String code, String message, HttpStatus status) {
        this.code = code;
        this.message = message;
        this.status = status;
    }

    public String getCode() {
        return code;
    }

    public String getMessage() {
        return message;
    }

    public HttpStatus getStatus() {
        return status;
    }
}

```

9. Exception cho từng tầng

9.1. BusinessException

Dùng cho lỗi nghiệp vụ ở application service.

```

package com.example.common.exception;

public class BusinessException extends BaseException {

    public BusinessException(ErrorCode errorCode) {
        super(errorCode);
    }

    public BusinessException(ErrorCode errorCode, String message) {
        super(errorCode, message);
    }

    public BusinessException(ErrorCode errorCode, String message, Throwable
cause) {
        super(errorCode, message, cause);
    }
}

```

```
}  
}
```

9.2. DomainException

Dùng cho domain rule/invariant.

```
package com.example.common.exception;  
  
public class DomainException extends BaseException {  
  
    public DomainException(ErrorCode errorCode) {  
        super(errorCode);  
    }  
  
    public DomainException(ErrorCode errorCode, String message) {  
        super(errorCode, message);  
    }  
}
```

Ví dụ:

```
package com.example.order.domain.exception;  
  
import com.example.common.exception.DomainException;  
import com.example.common.exception.ErrorCode;  
  
public class OrderAlreadyPaidException extends DomainException {  
  
    public OrderAlreadyPaidException(Long orderId) {  
        super(  
            ErrorCode.ORDER_ALREADY_PAID,  
            "Order " + orderId + " has already been paid"  
        );  
    }  
}
```

9.3. ResourceNotFoundException

```
package com.example.common.exception;

public class ResourceNotFoundException extends BaseException {

    public ResourceNotFoundException(ErrorCode errorCode, String message) {
        super(errorCode, message);
    }
}
```

Ví dụ:

```
package com.example.order.domain.exception;

import com.example.common.exception.ErrorCode;
import com.example.common.exception.ResourceNotFoundException;

public class OrderNotFoundException extends ResourceNotFoundException {

    public OrderNotFoundException(Long orderId) {
        super(
            ErrorCode.ORDER_NOT_FOUND,
            "Order not found with id: " + orderId
        );
    }
}
```

9.4. ExternalServiceException

Dùng trong infrastructure/client layer.

```
package com.example.common.exception;

public class ExternalServiceException extends BaseException {

    private final String serviceName;

    public ExternalServiceException(String serviceName, Throwable cause) {
        super(
            ErrorCode.EXTERNAL_SERVICE_ERROR,
            "External service error: " + serviceName,
            cause
        );
    }
}
```

```

        cause
    );
    this.serviceName = serviceName;
}

public ExternalServiceException(ErrorCode errorCode, String serviceName,
    Throwable cause) {
    super(
        errorCode,
        errorCode.getMessage() + ": " + serviceName,
        cause
    );
    this.serviceName = serviceName;
}

public String getServiceName() {
    return serviceName;
}
}

```

9.5. ExternalServiceTimeoutException

```

package com.example.common.exception;

public class ExternalServiceTimeoutException extends BaseException {

    private final String serviceName;

    public ExternalServiceTimeoutException(String serviceName, Throwable cause)
    {
        super(
            ErrorCode.EXTERNAL_SERVICE_TIMEOUT,
            "External service timeout: " + serviceName,
            cause
        );
        this.serviceName = serviceName;
    }

    public String getServiceName() {
        return serviceName;
    }
}

```

9.6. FileProcessingException

```
package com.example.common.exception;

public class FileProcessingException extends BaseException {

    public FileProcessingException(ErrorCode errorCode, String message,
    Throwable cause) {
        super(errorCode, message, cause);
    }

    public static FileProcessingException readFailed(Throwable cause) {
        return new FileProcessingException(
            ErrorCode.FILE_READ_ERROR,
            "Cannot read uploaded file",
            cause
        );
    }

    public static FileProcessingException invalidFormat(Throwable cause) {
        return new FileProcessingException(
            ErrorCode.FILE_FORMAT_INVALID,
            "Invalid file format",
            cause
        );
    }
}
```

9.7. DatabaseOperationException

```
package com.example.common.exception;

public class DatabaseOperationException extends BaseException {

    public DatabaseOperationException(String message, Throwable cause) {
        super(ErrorCode.DATABASE_ERROR, message, cause);
    }
}
```

10. Controller Layer

Controller chỉ nên:

- Nhận request.
- Validate input bằng annotation.
- Gọi service.
- Trả ApiResponse thành công.
- Không xử lý try-catch nghiệp vụ.

Ví dụ:

```
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @PostMapping("/{id}/pay")
    public ResponseEntity<ApiResponse<OrderResponse>> payOrder(
        @PathVariable Long id
    ) {
        OrderResponse response = orderService.payOrder(id);

        return ResponseEntity.ok(ApiResponse.success(response));
    }

    @PostMapping("/import")
    public ResponseEntity<ApiResponse<Void>> importOrders(
        @RequestParam MultipartFile file
    ) {
        orderService.importOrders(file);

        return ResponseEntity.ok(
            ApiResponse.success("Import completed", null)
        );
    }
}
```

Không nên:

```
@PostMapping("/{id}/pay")
public ResponseEntity<?> payOrder(@PathVariable Long id) {
    try {
```



```

        return ResponseEntity.ok(orderService.payOrder(id));
    } catch (OrderAlreadyPaidException e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body("Error");
    }
}

```

11. Application Service Layer

Service xử lý use case và transaction.

```

@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentClient paymentClient;

    @Transactional
    public OrderResponse payOrder(Long orderId) {
        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException(orderId));

        order.pay();

        PaymentResult paymentResult = paymentClient.charge(order);

        if (!paymentResult.isSuccess()) {
            throw new BusinessException(
                ErrorCode.EXTERNAL_SERVICE_ERROR,
                "Payment failed"
            );
        }

        order.markPaid();
        orderRepository.save(order);

        return OrderResponse.from(order);
    }
}

```

Ở đây:

- `OrderNotFoundException` là not found.
 - `order.pay()` có thể throw domain exception.
 - `paymentClient.charge()` nếu lỗi network sẽ throw `ExternalServiceException`.
 - Transaction rollback vì custom exception extends `RuntimeException`.
-

12. Domain Layer

Domain entity tự bảo vệ rule nội bộ.

```
public class Order {  
  
    private Long id;  
    private OrderStatus status;  
  
    public void pay() {  
        if (this.status == OrderStatus.PAID) {  
            throw new OrderAlreadyPaidException(this.id);  
        }  
  
        if (this.status == OrderStatus.CANCELLED) {  
            throw new DomainException(  
                ErrorCode.ORDER_INVALID_STATE,  
                "Cancelled order cannot be paid"  
            );  
        }  
    }  
  
    public void markPaid() {  
        this.status = OrderStatus.PAID;  
    }  
}
```

Domain không trả HTTP status, không biết response, không biết controller.

13. Infrastructure Layer

Infrastructure catch lỗi kỹ thuật và wrap.

13.1. PaymentClient

```
@Component
@RequiredArgsConstructor
public class PaymentClient {

    private final WebClient webClient;

    public PaymentResult charge(Order order) {
        try {
            return webClient.post()
                .uri("/payments/charge")
                .bodyValue(toPaymentRequest(order))
                .retrieve()
                .bodyToMono(PaymentResult.class)
                .block();
        } catch (WebClientRequestException e) {
            throw new ExternalServiceTimeoutException("PAYMENT_SERVICE", e);
        } catch (WebClientResponseException.ServiceUnavailable e) {
            throw new ExternalServiceException(
                ErrorCode.EXTERNAL_SERVICE_UNAVAILABLE,
                "PAYMENT_SERVICE",
                e
            );
        } catch (WebClientException e) {
            throw new ExternalServiceException("PAYMENT_SERVICE", e);
        }
    }

    private PaymentRequest toPaymentRequest(Order order) {
        return new PaymentRequest(order.getId());
    }
}
```

13.2. File parser

```
@Component
public class OrderCsvParser {

    public List<OrderImportRow> parse(MultipartFile file) {
```

```

        try (InputStream inputStream = file.getInputStream()) {
            // parse CSV
            return List.of();
        } catch (IOException e) {
            throw FileProcessingException.readFailed(e);
        } catch (RuntimeException e) {
            throw FileProcessingException.invalidFormat(e);
        }
    }
}

```

13.3. Repository/JDBC

Nếu dùng Spring Data JPA, Spring đã wrap nhiều lỗi database thành `DataAccessException`.

```

@Repository
@RequiredArgsConstructor
public class CustomOrderRepository {

    private final JdbcTemplate jdbcTemplate;

    public OrderSummary findSummary(Long orderId) {
        try {
            return jdbcTemplate.queryForObject(
                "select id, total_amount from orders where id = ?",
                this::mapRow,
                orderId
            );
        } catch (EmptyResultDataAccessException e) {
            throw new OrderNotFoundException(orderId);
        } catch (DataAccessException e) {
            throw new DatabaseOperationException(
                "Cannot query order summary",
                e
            );
        }
    }

    private OrderSummary mapRow(ResultSet rs, int rowNum) throws SQLException {
        return new OrderSummary(
            rs.getLong("id"),

```

```
        rs.getBigDecimal("total_amount")
    );
}
}
```

14. GlobalExceptionHandler

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(BaseException.class)
    public ResponseEntity<ErrorResponse> handleBaseException(
        BaseException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ex.getErrorCode();
        HttpStatus status = errorCode.getStatus();

        logByStatus(status, errorCode, ex, request);

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            ex.getMessage(),
            request.getRequestURI(),
            MDC.get("traceId")
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
        MethodArgumentNotValidException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.VALIDATION_ERROR;
        HttpStatus status = errorCode.getStatus();

        List<FieldErrorResponse> details = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(FieldErrorResponse::new)
            .collect(Collectors.toList());

        return ResponseEntity.status(status).body(
            ErrorResponse.of(status.value(), status.name(), errorCode.getCode(),
                ex.getMessage(), request.getRequestURI(), MDC.get("traceId"), details)
        );
    }
}
```

```

        .map(fieldError -> new FieldErrorResponse(
            fieldError.getField(),
            fieldError.getDefaultMessage(),
            fieldError.getRejectedValue()
        ))
        .toList();

ErrorResponse response = ErrorResponse.validation(
    status.value(),
    status.name(),
    errorCode.getCode(),
    errorCode.getMessage(),
    request.getRequestURI(),
    MDC.get("traceId"),
    details
);

return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleUnexpected(
    Exception ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.INTERNAL_SERVER_ERROR;
    HttpStatus status = errorCode.getStatus();

    log.error(
        "Unexpected exception. path={}, traceId={}",
        request.getRequestURI(),
        MDC.get("traceId"),
        ex
    );

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        errorCode.getMessage(),
        request.getRequestURI(),
        MDC.get("traceId")
    );

    return ResponseEntity.status(status).body(response);
}

private void logByStatus(

```

```

        HttpStatus status,
        ErrorCode errorCode,
        Exception ex,
        HttpServletRequest request
    ) {
        if (status.is5xxServerError()) {
            log.error(
                "Server error. code={}, path={}, traceId={}",
                errorCode.getCode(),
                request.getRequestURI(),
                MDC.get("traceId"),
                ex
            );
        } else {
            log.warn(
                "Handled exception. code={}, path={}, traceId={}, message={}",
                errorCode.getCode(),
                request.getRequestURI(),
                MDC.get("traceId"),
                ex.getMessage()
            );
        }
    }
}

```

15. ErrorResponse

```

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private boolean success;
    private Instant timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private String path;
    private String traceId;
    private List<FieldErrorResponse> details;

    public static ErrorResponse of(
        int status,
        String error,
        String code,

```

```

        String message,
        String path,
        String traceId
    ) {
        ErrorResponse response = new ErrorResponse();
        response.success = false;
        response.timestamp = Instant.now();
        response.status = status;
        response.error = error;
        response.code = code;
        response.message = message;
        response.path = path;
        response.traceId = traceId;
        return response;
    }

    public static ErrorResponse validation(
        int status,
        String error,
        String code,
        String message,
        String path,
        String traceId,
        List<FieldErrorResponse> details
    ) {
        ErrorResponse response = of(status, error, code, message, path,
        traceId);
        response.details = details;
        return response;
    }

    // getter/setter
}

```

16. Request traceId

```

@Component
public class RequestIdFilter extends OncePerRequestFilter {

    private static final String TRACE_ID = "traceId";
    private static final String TRACE_ID_HEADER = "X-Trace-Id";

    @Override
    protected void doFilterInternal(

```



```

        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) throws ServletException, IOException {

        String traceId = request.getHeader	TRACE_ID_HEADER);

        if (traceId == null || traceId.isBlank()) {
            traceId = UUID.randomUUID().toString().replace("-", "");
        }

        try {
            MDC.put	TRACE_ID, traceId);
            response.setHeader	TRACE_ID_HEADER, traceId);
            filterChain.doFilter(request, response);
        } finally {
            MDC.remove	TRACE_ID);
        }
    }
}

```

17. Mapping exception theo tầng

Layer	Exception gốc	Wrap thành	HTTP Status
Controller	MethodArgumentNotValidException	Validation error	422
Controller	HttpMessageNotReadableException	Invalid request	400
Service	Business rule fail	BusinessException	400 / 409
Service	Resource missing	ResourceNotFoundException	404
Domain	Invalid state	DomainException	400 / 409
Infrastructure	IOException	FileProcessingException	400 / 500
Infrastructure	External timeout	ExternalServiceTimeoutException	504
Infrastructure	External 5xx	ExternalServiceException	502 / 503
Infrastructure	DataAccessException	DatabaseOperationException	500

Layer	Exception gốc	Wrap thành	HTTP Status
Unknown	Unexpected bug	<code>Exception.class</code> fallback	500

18. Transaction rule

Spring `@Transactional` mặc định rollback với:

```
RuntimeException
Error
```

Không rollback mặc định với checked exception.

Vì vậy production nên:

```
IOException / SQLException / SDK checked exception
↓
wrap thành custom RuntimeException
↓
transaction rollback tự nhiên
```

Ví dụ:

```
@Transactional
public void importOrders(MultipartFile file) {
    List<OrderImportRow> rows = orderCsvParser.parse(file);
    orderRepository.saveAll(toEntities(rows));
}
```

Nếu `orderCsvParser.parse(file)` throw `FileProcessingException` extends `RuntimeException`, transaction rollback.

Chỉ dùng `rollbackFor` khi bắt buộc giữ checked exception:

```
@Transactional(rollbackFor = IOException.class)
public void importOrders(MultipartFile file) throws IOException {
    // processing
}
```

19. Logging rule theo tầng

Controller

Thông thường không log exception ở controller.

Service

Chỉ log business context nếu cần, không log stack trace trùng lặp.

```
log.info("Start paying order. orderId={}, traceId={}", orderId,
MDC.get("traceId"));
```

Infrastructure

Có thể log context kỹ thuật nếu quan trọng, nhưng tránh log stack trace nhiều lần.

```
log.warn("Payment service timeout. service={}, orderId={}", "PAYMENT_SERVICE",
orderId);
```

GlobalExceptionHandler

Là nơi log chính:

- 4xx: WARN hoặc INFO .
- 5xx: ERROR kèm stack trace.
- Unexpected exception: ERROR .

20. Những lỗi thiết kế thường gặp

20.1. Catch exception ở controller

Không nên:

```
try {
    service.doSomething();
} catch (Exception e) {
```

```
        return ResponseEntity.badRequest().body(e.getMessage());
    }
```

Vì controller bị bẩn và response không thống nhất.

20.2. Throw RuntimeException chung chung

Không nên:

```
throw new RuntimeException("Order not found");
```

Nên:

```
throw new OrderNotFoundException(orderId);
```

20.3. Infrastructure exception leak lên service/controller

Không nên:

```
public PaymentResult charge(Order order) throws IOException {
}
```

Nên:

```
public PaymentResult charge(Order order) {
    try {
        // call external
    } catch (IOException e) {
        throw new ExternalServiceException("PAYMENT_SERVICE", e);
    }
}
```

20.4. Domain exception phụ thuộc HTTP

Không nên:

```
throw new ResponseStatusException(HttpStatus.CONFLICT, "Order paid");
```

trong domain/service.

Nên dùng:

```
throw new OrderAlreadyPaidException(orderId);
```

20.5. Log ERROR cho mọi exception

Không nên log ERROR cho validation/business expected error.

Validation fail, not found thông thường, business conflict không nhất thiết là system error.

20.6. Trả stack trace/message gốc cho client

Không nên:

```
{
  "message": "could not execute statement; constraint [uk_user_email]"
}
```

Nên:

```
{
  "code": "DB_409",
  "message": "Duplicate resource"
}
```

21. Checklist production

Controller Layer

- Không try-catch business exception.
- Không throw checked exception kỹ thuật.
- Validate request bằng annotation.

- Return `ApiResponse<T>` cho success.

Service Layer

- Throw business exception rõ ràng.
- Không leak SDK/IO/SQL exception.
- Quản lý transaction.
- Custom exception extends RuntimeException để rollback đúng.

Domain Layer

- Bảo vệ invariant.
- Không phụ thuộc HTTP/Spring.
- Không gọi repository/client trực tiếp.
- Exception nên thể hiện rule nghiệp vụ.

Infrastructure Layer

- Catch lỗi kỹ thuật.
- Wrap thành application exception.
- Preserve root cause.
- Không leak provider exception.
- Phân biệt timeout, unavailable, bad gateway.

Global Handler

- Map exception sang ErrorResponse.
- Có traceId.
- Có HTTP status đúng.
- Có errorCode.
- Không expose internal details.
- Log đúng level.

22. Câu trả lời phỏng vấn mẫu

Trong dự án production, em thiết kế exception theo phân tầng thay vì throw/catch tùy tiện ở mọi nơi. Controller chỉ nhận request, validate input và trả response thành công, không catch business exception trực tiếp. Những lỗi như validation fail, request body sai JSON, thiếu param sẽ được xử lý tập trung trong `GlobalExceptionHandler`.

Ở service layer, em xử lý use case và business rule. Nếu có lỗi nghiệp vụ như order đã thanh toán, trạng thái không hợp lệ hoặc resource không tồn tại, em throw custom exception như `BusinessException`, `OrderAlreadyPaidException`, `ResourceNotFoundException`. Các exception này đều extends từ `BaseException`, có `ErrorCode` và HTTP status tương ứng.

Ở domain layer, em để entity hoặc domain service tự bảo vệ business invariant. Ví dụ `Order.pay()` sẽ throw `OrderAlreadyPaidException` nếu order đã ở trạng thái `PAID`. Domain layer không phụ thuộc Spring, không biết `HttpStatus`, không trả `ResponseEntity`.

Ở infrastructure layer, em xử lý các lỗi kỹ thuật như `IOException`, `SQLException`, `WebClientException`, `FeignException`, SDK exception từ payment/email/storage. Các lỗi này không được leak lên controller mà được catch và wrap thành custom runtime exception như `ExternalServiceException`, `ExternalServiceTimeoutException`, `FileProcessingException`, `DatabaseOperationException`. Việc wrap này cũng giúp transaction rollback đúng vì Spring mặc định rollback với `RuntimeException`.

Cuối cùng, `GlobalExceptionHandler` là nơi duy nhất convert exception thành `ErrorResponse` thống nhất gồm `success`, `timestamp`, `status`, `error`, `code`, `message`, `path`, `traceId`. Với lỗi 4xx như validation hoặc business conflict thì log ở mức INFO/WARN. Với lỗi 5xx như database, external service hoặc unexpected exception thì log ERROR kèm stack trace. Response trả ra client luôn an toàn, không expose stack trace, SQL, internal URL hoặc exception message gốc từ SDK.

Tóm lại, phân tầng exception chuẩn production giúp code sạch hơn, transaction an toàn hơn, response nhất quán hơn và production incident dễ debug hơn nhờ `traceId` và `errorCode` rõ ràng.

23. Kết luận

Thiết kế phân tầng exception chuẩn production nên đi theo hướng:

Controller

- Không catch business exception
- Validate request

Service

- Throw business/application exception
- Quản lý transaction

Domain

- Throw domain exception
- Bảo vệ business invariant

Infrastructure

- Catch technical exception
- Wrap thành application exception

GlobalExceptionHandler

- Convert exception thành ErrorResponse
- Log, traceId, HTTP status, errorCode

Nguyên tắc quan trọng nhất:

Không để lỗi kỹ thuật leak lên tầng trên.
Không để controller xử lý lỗi thủ công.
Không làm mất root cause.
Không trả lỗi nội bộ ra client.
Không dùng RuntimeException chung chung.

Cách thiết kế này phù hợp với Spring Boot monolith, modular monolith và microservices.